# Theoretical Question

Your Name

November 22, 2025

## 1 Problem Definition

Let us denote the HW problem as HEP (Hurricane Evacuation Problem). Let us also denote the inputs to the problem:

- $G = (V, E, w)$: The input graph, where $V$ is the set of vertices (locations), $E$ is the set of edges (roads), and $w : E \to \mathbb{Z}^+$ is a function mapping each edge to a positive integer weight (travel time).
- $S \subseteq V$: The set of target vertices that contain people to be rescued.
- $F \subseteq E$: The set of "flooded" edges.
- $K \subseteq V$: The set of vertices that contain an amphibian kit.
- $v_{\text{start}} \in V$: A designated starting vertex for the agent.
- $Q \in \mathbb{Z}^+$: A constant time cost to equip an amphibian kit.
- $U \in \mathbb{Z}^+$: A constant time cost to unequip an amphibian kit.
- $P \in \mathbb{Z}^+, P > 1$: An integer speed penalty factor. When a kit is equipped, all travel time is multiplied by $P$.
- $T \in \mathbb{Z}^+$: A maximum time bound.

We also have the following set of actions:

- Traverse($e$) for $e \in E$: Traverse edge $e$.
  - If $e \notin F$ (not flooded): Always succeeds, costs $w(e)$ time if unequipped, $P \cdot w(e)$ if equipped.
  - If $e \in F$ (flooded): Succeeds only if equipped (costs $P \cdot w(e)$); otherwise acts as NoOp.
- Equip: Equip an amphibian kit at the current vertex (only valid if kit is present). Costs $Q$.
- Unequip: Unequip the amphibian kit at the current vertex (only valid if currently equipped). Costs $U$. The kit remains at this vertex.

- NoOp: Do nothing. Costs 1.

We also have the following rules:

- Upon arriving at a vertex $v$, the agent automatically picks up all people at $v$.
- An agent can be equipped with at most one kit at a time.
- A kit can only be equipped if present at the current vertex and the agent is not already equipped.

We define the Hurricane Evacuation Problem as follows. Given an instance $I = (G, S, F, K, v_{\text{start}}, Q, U, P, T)$, the problem asks whether there is a way to rescue all people in $S$, starting from $v_{\text{start}}$, using a sequence of valid actions, in time at most $T$.

We shall prove that HEP $\in$ NP.

# 2 Proving HEP $\in$ NP

*Proof.* To prove HEP $\in$ NP, we need to show that there exists a certificate that can be verified in polynomial time.

Let us define the certificate as $C = (a_1, a_2, \ldots, a_k)$ ,i.e C is a sequence of actions such that each action $a_i$ is one of the existing valid actions:

$$a_i \in \{\textsc{Traverse}(e) : e \in E\} \cup \{\textsc{Equip}, \textsc{Unequip}, \textsc{NoOp}\} \tag{1}$$

Moreover,each action shall be encoded in the certificate. For a $\textsc{Traverse}(e)$ action, we specify which edge $e \in E$ is being traversed which requiring $\lceil \log_2 |E| \rceil$ bits. The remaining actions Equip, Unequip, and NoOp need only a constant number of bits. Therefore we get that each action requires $O(\log |E|)$ bits to be encoded.

Since the sequence contains in total $k$ actions, and since we have a time limit,so the certificate size does not grow exponentially - it remains polynomial in the input size. Thus, the certificate size is $O(k \log |E|)$ bits

**Verification algorithm.** Let us now describe the verification algorithm. The verification alogirithm will works will take the certificate which contains a sequence of actions and checking whether this results in all people to be rescued within the time bound. Algorithm 1 presents this procedure:

**Algorithm 1** Verify-HEP$(I, C)$

**Require:** Instance $I = (G, S, F, K, v_{\text{start}}, Q, U, P, T)$
**Require:** Certificate $C = (a_1, a_2, \ldots, a_k)$
**Ensure:** Returns `ACCEPT` or `REJECT`

{Initialize state}
$v_{\text{current}} \leftarrow v_{\text{start}}$
equipped $\leftarrow$ `false`
rescued $\leftarrow \emptyset$
total_time $\leftarrow 0$

**for** $i = 1$ to $k$ **do**
  **if** $a_i = \textsc{Traverse}(e)$ where $e = (u, w)$ **then**
    **if** $u \neq v_{\text{current}}$ **and** $w \neq v_{\text{current}}$ **then**
      **return** `REJECT`
    **end if**

    **if** $e \in F$ **and** equipped $=$ `false` **then**
      total_time $\leftarrow$ total_time $+ 1$
    **else**
      $v_{\text{next}} \leftarrow \begin{cases} w & \text{if } u = v_{\text{current}} \\ u & \text{if } w = v_{\text{current}} \end{cases}$

      **if** equipped $=$ `true` **then**
        total_time $\leftarrow$ total_time $+ P \cdot w(e)$
      **else**
        total_time $\leftarrow$ total_time $+ w(e)$
      **end if**

      $v_{\text{current}} \leftarrow v_{\text{next}}$
      rescued $\leftarrow$ rescued $\cup \{v_{\text{current}}\}$
    **end if**

  **else if** $a_i = \textsc{Equip}$ **then**
    **if** $v_{\text{current}} \notin K$ **or** equipped $=$ `true` **then**
      **return** `REJECT`
    **end if**
    equipped $\leftarrow$ `true`
    total_time $\leftarrow$ total_time $+ Q$

  **else if** $a_i = \textsc{Unequip}$ **then**
    **if** equipped $=$ `false` **then**
      **return** `REJECT`
    **end if**
    equipped $\leftarrow$ `false`
    total_time $\leftarrow$ total_time $+ U$

  **else if** $a_i = \textsc{NoOp}$ **then**
    total_time $\leftarrow$ total_time $+ 1$
  **end if**
**end for**

**if** $S \subseteq$ rescued **and** total_time $\leq T$ **then**
  **return** `ACCEPT`
**else**
  **return** `REJECT`
**end if**

The algorithm starts at the initial node $v_{\text{start}}$ and runs each action in sequence. For each action, it first validates that the action is legal , if the check fails then the algorithm rejects.

After valdiation we run the action and update the state accordigny.

After all actions have been processed, we verify that all people in $S$ have been rescued and that the total time does not exceed $T$. If both conditions are satisfied, we accept the certificate.

**Time complexity** The initialization takes $O(1)$ . We run the main loop k times. For each iteration of the look we do constant number of action action in O(1) and so this part runs in O(k).

Then we check if $S \subseteq$ rescued, which takes $O(|S|)$ time.Then we also do constant-time comparison of the total time against $T$ in O(1).So in total: $O(1)+O(k)+O(|S|) = O(k+|S|)$

So we get that k in polynomial in the size of the input.

**Correctness**

We shall now prove correctness. Let us prove completeness.

**1. completeness**

Let us assume that there exists a valid sequence of actions $C^* = (a_1^*, \ldots, a_k^*)$ such that all the people in $S$ are rescued within time $T$.

Let us denote this sequence $C^*$ as our certificate. After the verifier processes $C^*$, each action will pass the validity checks since $C^*$ is a valid solution( by assumption). The simulation will execute correctly and this will result in all the people being rescued in the required time. Thus, the final verification step will succeed and the verifier will accept the certificate. So when a solution exists there is a certificate that will be accepted.

Let us now prove soundness.

**2. soundness**

Let us suppose that the verifier accepts some certificate $C = (a_1, \ldots, a_k)$. So when the algorithm reached the final acceptance without rejecting any action, and both final conditions were satisfied. Therefore all actions in $C$ passed their validity checks, all people in $S$ were added to the rescued set, and the total time was at most $T$. This means that $C$ represents a valid solution to HEP.

We have therefore shown that the certificate has polynomial size, can be verified in polynomial time, and the verifier is both complete and sound. Therefore, by the definition of the complexity class NP, we conclude that HEP $\in$ NP. $\square$

# 3 Proving HEP $\in$ NP-Hard

To show that HEP is NP-Hard, we will show a reduction from the **Hamiltonian Path Problem** (fixed start vertex) to HEP. Note that The Hamiltonian Path Problem gets as input graph $G' = (V', E')$ and a start vertex $u \in V'$ and returns if it has a hamiltion path starting from this node

*Proof.* **The Reduction.** Given an HPP instance $(G', u)$, we construct an HEP instance $I$ as follows:

- $G = G'$, but we assign weight $w(e) = 1$ to every edge $e \in E'$.
- $S = V'$ (we must rescue people from every vertex).
- $v_{\text{start}} = u$.
- $F = \emptyset, K = \emptyset$ (no flooding, no kits).
- $Q = 0, U = 0, P = 1$.
- $T = |V'| - 1$.

**Correctness.** ($\Rightarrow$) Suppose $G'$ has a Hamiltonian Path starting at $u$. This path contains exactly $|V'|$ vertices and $|V'| - 1$ edges. Since every edge has weight 1, the total time to traverse this path is $|V'| - 1$. This rescues all nodes in $S$ within time $T$. Thus, HEP accepts.

($\Leftarrow$) Suppose HEP has a solution. The agent visits all $|V'|$ vertices in time $T \leq |V'| - 1$. To visit $|V'|$ distinct vertices, the agent must traverse at least $|V'| - 1$ edges. Since each edge costs 1, the minimum time to visit all vertices is $|V'| - 1$. Since the solution found occurs in exactly (or at most) $|V'| - 1$, the agent must not have wasted any steps revisiting vertices. The path must be a simple path visiting every node exactly once. Thus, a Hamiltonian Path exists. $\square$